DAY 09 · MOCK EXAM 1

Mock Exam #1

A full timed rehearsal: broken BPE tokenizer, 51 tests.

120 min + review / day09_mock_exam_1 / Timed exam

INSIDE THIS BOOKLET

BPE · Timed run · Debrief · Bug journal

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day09_mock_exam_1/` · the course runner `python run.py 9`

In this session

Rules 3

Before the timer: background you need 3

Start 6

After the timer 7

End-of-day checklist 7

> ◎ **The brief**
>
> A 120-minute dress rehearsal of the real assessment: fix a broken BPE tokenizer and maximize the 51 passing tests. No AI, no web, no solutions.

Rules

- **Set a 120-minute timer.** Close everything except an editor, a terminal, and `exam/`.
- No AI, no web search, no looking at `solution/`. `help()` and `dir()` in a Python REPL are allowed.
- Only edit `exam/bpe.py`. Don't touch the tests.
- Your score = number of passing tests out of 51.

Why be so strict with yourself? Today's real purpose is to find out how *you* perform under exam conditions: how far you get in two hours, and which kinds of bugs cost you the most time. A quick peek at the solution or a half-hour break gives you a score that tells you nothing. Most of the learning happens afterwards, in the review, and the review is only useful if the attempt was honest.

Before the timer: background you need

You're allowed to read this section before starting the timer. You don't need any knowledge of machine learning: a tokenizer is a text-processing algorithm, built entirely from things you've practised (strings on Day 2, dicts on Day 3, classes on Day 4, bytes and UTF-8 on Day 5).

0.1 What a tokenizer is

Language models like the ones behind chat assistants don't read letters. They read sequences of whole numbers, called **tokens**. A **tokenizer** is the program that converts text into tokens (*encoding*) and tokens back into text (*decoding*). For example, `"hello world"` might become `[31373, 995]`. There's a trade-off in choosing what a token stands for:

- **One token per byte.** There are only 256 possible tokens, and *any* text can be represented, including typos, emoji and unseen languages. But sequences get very long: `"the"` takes 3 tokens.
- **One token per word.** Sequences are short, but the vocabulary is enormous, and a word that wasn't in the vocabulary (a new name, a typo) can't be represented at all.

Byte Pair Encoding (BPE) gets the best of both. It starts from single bytes, so anything can always be represented, and then *learns* extra tokens for byte sequences that occur often. Common words and pieces of words end up as single tokens, while rare text falls back to smaller pieces.

0.2 Training: how the extra tokens are learned

Training looks at a sample text and repeatedly finds the most common pair of neighbouring tokens, then gives that pair a token of its own:

1. Start with the UTF-8 bytes of the text, as numbers 0–255. These are the first 256 tokens.
2. **Count** how often each pair of neighbouring tokens occurs (the function `get_stats`).
3. Take the **most frequent** pair, with a fixed rule for ties.
4. Give it a **new id**: 256 for the first pair learned, then 257, 258, and so on. Record this **merge**: "`(a, b)` becomes `new_id`".

5. **Replace** every occurrence of the pair in the sequence with the new id (the function `merge`).
6. Repeat from step 2 until you have as many tokens as requested, or no pair occurs at least twice.

Each new token stands for the bytes of its two parts glued together: `vocab[new_id] = vocab[a] + vocab[b]`.

0.3 A worked example of training

Let's train on the text `"aaabdaaabac"`. In UTF-8 the letters are single bytes: a = 97, b = 98, c = 99, d = 100. To keep it readable, the table writes letters instead of numbers.

Step	Current sequence	Most common pairs	Chosen pair	New token
start	a a a b d a a a b a c	(a,a) ×4, (a,b) ×2	(a,a)	256 = aa
1	256 a b d 256 a b a c	(256,a) ×2 and (a,b) ×2: a tie	(a,b), the smaller pair, since 97 < 256	257 = ab
2	256 257 d 256 257 a c	(256,257) ×2	(256,257)	258 = aaab
3	258 d 258 a c	every pair occurs once	none: stop	—

Four details in this table are spec rules, and each is a likely bug location:

- **Counting overlaps.** In `a a a`, the pair `(a, a)` is counted twice: positions 0–1 and 1–2. That's why the first row says ×4 (two in each `aaa`).
- **Replacing without overlaps.** When merging, `a a a` becomes `256 a`, not `256 256`. Once a pair has been replaced, the scan jumps past *both* of its elements (the A6 pattern from Day 7).
- **Tie-breaking.** In step 1, two pairs occur twice each. The spec says to choose the **smallest pair**, comparing tuples (Day 3): `(97, 98)` is smaller than `(256, 97)`.
- **Stopping.** In step 3 no pair occurs twice, so training stops, even if more tokens were requested. Merging a pair that occurs only once doesn't save anything.

0.4 Encoding new text: replaying the merges

Once trained, the tokenizer can encode any text. Convert it to bytes, then apply the learned merges. The order matters: at each step, among the pairs present in the sequence, apply the one that was **learned earliest**, i.e. the one with the lowest new id. That's because later merges were learned on top of earlier ones: 258 is made of 256 and 257, so those must exist first. Encoding `"aaab"` with the merges above:

Sequence	Pairs present that are known merges	Apply
a a a b	(a,a) → 256, (a,b) → 257	256, the lowest
256 a b	(a,b) → 257 (and (256,a), which isn't a merge)	257
256 257	(256,257) → 258	258
258	none	done

Result: `[258]`, a single token for the whole string.

0.5 Decoding: bytes first, then text

To decode, look up each token's bytes in `vocab`, **join all the bytes together**, and only then decode the result as UTF-8. As you saw on Day 5, a token can hold *part* of a multi-byte character: one token might be `\xc3` and the next `\xa9`. Decoding them separately would give `ÃÃ` instead of `é`. The spec also says to decode with `errors="replace"`, so a sequence of ids that really doesn't form valid UTF-8 produces `` instead of crashing.

0.6 The extra features in the spec

The spec adds three features on top of basic BPE:

- **Pre-splitting (`split_text`).** Before training or encoding, the text is cut into chunks: words, numbers, runs of whitespace and single punctuation characters, with a space attached to the front of the word that follows it. Merges are never allowed to cross a chunk boundary. That's why `" world"` can become a single token, but `"d!"` never will. The function is a scanning loop (Day 2) with several rules applied in priority order. Read its examples in the spec carefully, because each rule is a chance for an off-by-one.
- **Special tokens.** Strings like `"<|endoftext|>"` that get a reserved id of their own and are never split up. When they're allowed, the encoder first looks for them in the text, and encodes only the text *between* them in the normal way. If two special tokens could start at the same position, the **longest** one wins.
- **Saving and loading.** The tokenizer can write its merges and special tokens to a small text format, and read them back. The order of the merges matters (the k-th merge must get id 256 + k). A special token's text can itself contain spaces, which is why the spec says the string is "everything after the first space" on its line.

0.7 How the code is organised

`get_stats` and `merge` are the low-level helpers. `train` and `encode_ordinary` use both of them, plus `split_text`. `encode` uses `encode_ordinary`, and `decode` uses `vocab`. So a bug in `get_stats` or `merge` makes many tests fail at once, including tests of functions that are themselves correct. Following the triage rule from Day 6, start with the tests of the helpers (`TestGetStats`, `TestMerge`, `TestSplitText`), and work your way up.

0.8 Python you'll meet in `bpe.py`

Everything the code uses has come up during the course, but a few lines pack several ideas together. Make sure you can read each of these at a glance *before* the timer starts, so that during the exam you're hunting bugs rather than decoding syntax. **Pairs as dict keys.** Pair counts are stored in a dict whose keys are tuples: `counts[(104, 105)] = 3` means "the pair of ids 104, 105 appears 3 times". Tuples can be dict keys because they can't change (Day 3, section 5). `pair[0]` and `pair[1]` are its two ids. **A dict passed in to be filled.** `get_stats(ids, counts=None)` counts the pairs of one list. When given an existing dict, it *adds to it* instead of starting a new one, so calling it once per chunk with the same dict totals the pairs over all chunks. The first line, `counts = {} if counts is None else counts`, is the safe-default pattern from Day 4 (section 2 and section 10): never use `{}` itself as the default. **Bytes as a list of numbers.** `list("hi".encode("utf-8"))` is `[104, 105]`: looping over bytes gives integers (Day 5). Going back, `bytes([104])` is `b"h"`, and `vocab[a] + vocab[b]` glues two byte strings together, so the vocabulary entry of a merged token is the bytes of its two halves. **The vocabulary in one line.** `{i: bytes([i]) for i in range(256)}` is a dict comprehension (Day 3, section 6): ids 0 to 255 each map to the single byte with that value. **Picking the most frequent pair, with a tie rule.**

```
pair = max(stats, key=lambda p: (stats[p], -p[0], -p[1]))
```

`max` over a dict looks at its keys, here the pairs (Day 3, section 4). The key function scores each pair with a tuple, and tuples compare item by item (Day 1, section 6):

1. first by `stats[p]`: the highest count wins;
2. on equal counts, by `-p[0]`: since `max` picks the *largest* value, negating the first id makes the *smallest* first id win;
3. if that's equal too, by `-p[1]`: the smallest second id.

So the tuple says, in order: "most frequent, then smallest first id, then smallest second id". Compare it word by word with the tie rule in the spec. A missing minus sign, or the elements in the wrong order, changes which pair is chosen only when there's a tie, which is exactly the kind of bug that slips past most tests. **Picking the earliest learned merge.**

```
pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
```

`self.merges` maps each learned pair to its id, and earlier merges have smaller ids. So the pair with the smallest id is the one learned first. Pairs that were never learned aren't in `merges`, and `get` gives them `float("inf")`, infinity, which is larger than any id (drill 7 uses the same trick). They lose to any learned pair. If *every* pair is unknown, `min` returns one of them anyway, which is why the next line checks `if pair not in self.merges: break`. **Inverting a dict.** `{idx: s for s, idx in self.special_tokens.items()}` swaps keys and values: from `text -> id` to `id -> text`, so decoding can look tokens up by id. **`startswith` at a position.** `text.startswith(s, i)` asks whether `s` occurs in `text` *starting at position `i`*, without slicing. It's the same as `text[i:].startswith(s)`. **Nested unpacking.** `merges.items()` gives pairs like `((104, 105), 256)`: a key that is itself a pair, and a value. The loop

```
for (a, b), _ in sorted(self.merges.items(), key=lambda kv: kv[1]):
```

unpacks both levels at once: `(a, b)` takes the key apart, and `_` throws away the value. `key=lambda kv: kv[1]` sorts the items by their value, the id, so the merges come out in the order they were learned. **Unpacking a generator.** `a, b = (int(x) for x in line.split())` converts both fields of a line like `"104 105"` to integers and unpacks them. If the line doesn't have exactly two fields, the unpacking raises `ValueError`. **`@property` and `@classmethod`.** `vocab_size` is a property (Day 4, section 3): you write `tok.vocab_size` with no parentheses, and the method runs. `load` is a class method (Day 4, section 4): it's called on the class, `BPETokenizer.load(text)`, receives the class as `cls`, and builds a new object with `cls()`. **Reading a text format by slicing lines.** In `load`, `lines[0]` is the header, `lines[1]` the number of merges `n`, and `lines[2:2 + n]` exactly the `n` merge lines. After them, `lines[2 + n]` holds the number of special tokens `m`, and `lines[3 + n:3 + n + m]` are their lines. Draw the file with line numbers next to it and check every index. Off-by-ones are easy here. **Decoding all at once.** `b"".join(parts).decode("utf-8", errors="replace")` first glues every token's bytes together and decodes **once** (the "never decode in pieces" rule from Day 5). `errors="replace"` turns invalid bytes into `\ufffd` instead of raising.

Start

> **Keep a pristine copy**
>
> Before you start, keep a pristine copy so you can retry later:

```
cp exam/bpe.py exam/bpe.original.py
```

Then start the timer and run:

```
cd day09_mock_exam_1/exam
python -m unittest 2>&1 | tail -3              # baseline score
python -m unittest -v 2>&1 | grep -E "ERROR|FAIL"
```

(or from the course root: `python run.py 9`) The first command gives your starting score. The second lists every failing test by name, which is your to-do list for triage.

Read before you fix

Read `exam/SPEC.md` first. Spend the first ~10 minutes reading the spec and the code **before** fixing anything (the Day 6 workflow). Note that broken code can be slow or even loop forever: if a test run hangs, press Ctrl-C and run test classes one at a time, e.g. `python -m unittest test_bpe.TestMerge`.

After the timer

1. Write down your score and which bugs you found, in what order, and how long each took.
2. Run `python run.py 9 --solutions` to see the reference pass 51/51.
3. Read `solution/BUGS.md` (the list of planted bugs) and compare with yours. For every bug you missed, ask: *which test pointed at it, and what would have led me to it faster?* Add it to your bug journal.
4. If you didn't finish, **reset and redo the remaining bugs** (`git`-less reset: keep a copy of the original — see below).

The review is worth as much as the attempt itself. When you look at the bugs you missed, you'll usually find a pattern: a category from the Day 7 catalog you don't check by reflex yet, a part of the spec you skimmed, or a lot of time lost on one stubborn test. That pattern is what to work on before Day 10. To reset, copy the pristine version back: `cp exam/bpe.original.py exam/bpe.py`.

End-of-day checklist

Tick every box

- ☐ I completed the 120-minute run with no AI, web search, or solutions.
- ☐ I recorded my score and the order in which I found the bugs.
- ☐ I read `solution/BUGS.md` and logged every bug I missed.

Score: _____ / 51 **Bugs found:** _____

Exam lens

- Read the spec before touching code, and baseline the score before your first edit.
- Fix high-leverage helpers; one correct fix can turn many tests green.
- Debrief every missed bug: the pattern matters more than the score.